

SouthPark API

Feld	Angabe
Projektname	SouthPark API
Projektart	Spring-Boot-REST-API mit 3-Schichten-Architektur, JPA und PostgreSQL
Teammitglieder	Silvio Apa, Nicola Schmied
Klasse	UIFZ 2524-021
Lehrperson	Graziano Laveder
Abgabedatum	11.07.2026
GitHub-Repository	https://github.com/silvio-apa/SouthPark-API



Inhaltsverzeichnis

1. Projektidee
2. Anforderungskatalog
3. Klassendiagramm / Datenmodell / Architektur
4. API-Abläufe je User Story
5. DTOs / Datentypen / JSON-Beispiele
6. REST-Schnittstellen, Validierung und Fehlerbehandlung
7. Testplan
8. Testergebnisse
9. Automatisierte Tests und JavaDoc
10. Installationsanleitung
11. Hilfestellungen / Quellen
12. Projektverlauf / Git-Pushes
13. Fazit
14. Arbeitsaufteilung

1. Projektidee

1.1 Ausgangslage und Problemstellung

Für South-Park-Charaktere und deren Zitate wird eine strukturierte, dauerhaft gespeicherte Datenbasis benötigt. Ohne Backend müssten die Daten lokal, unstrukturiert oder nur temporär verwaltet werden. Die SouthPark API löst dieses Problem, indem Characters und Quotes über klar definierte HTTP-Endpunkte erstellt, gelesen, aktualisiert und gelöscht werden können. Eine REST-API ist dafür sinnvoll, weil unterschiedliche Clients die Funktionen unabhängig von einer Benutzeroberfläche über standardisierte HTTP-Methoden und JSON nutzen können. Die Speicherung in PostgreSQL stellt sicher, dass die Daten auch nach einem Neustart erhalten bleiben.

1.2 Projektidee

Die SouthPark API ist ein Spring-Boot-Backend zur Verwaltung von South-Park-Charakteren und ihren Zitaten. Zu einem Character werden Name, Alter, Geschlecht, Tätigkeit, Status und eine Bild-URL gespeichert. Jedem Character können beliebig viele Quotes mit Text, Staffel und Episode zugeordnet werden. Die Anwendung stellt vollständiges CRUD für Characters und Quotes bereit und bildet zwischen beiden Entities eine echte 1:n-Beziehung ab. Controller nehmen HTTP-Anfragen entgegen, Services koordinieren die Anwendungslogik und Repositories greifen über Spring Data JPA auf PostgreSQL zu. Nach aussen werden ausschliesslich DTOs ausgeliefert. Bean Validation prüft Eingaben, während ein GlobalExceptionHandler strukturierte Fehlerantworten für ungültige Daten und nicht gefundene Ressourcen erzeugt. Damit eignet sich die API als stabiles Backend für REST-Clients, spätere Frontends und technische Tests.

1.3 Zielgruppe und Nutzen

Zielgruppe	Bedürfnis / Problem	Nutzen der API
Frontend- oder API-Entwickelnde	Benötigen eine klar definierte Schnittstelle für Character- und Quote-Daten.	Standardisierte JSON-Endpunkte mit CRUD-Funktionen und DTOs.
Lehrperson und Projektteam	Benötigen einen nachvollziehbaren, testbaren Backend-Prozess mit Persistenz.	Reproduzierbare Installation, strukturierte Architektur sowie manuelle und automatisierte Tests.

1.4 Projektziele und Abgrenzung

Kategorie	Beschreibung
Muss-Ziele	Vollständiges Character-CRUD; Quote-CRUD; 1:n-Beziehung; DTOs und Mapper; Bean Validation; strukturierte 400-/404-Fehler; PostgreSQL via Docker; Tests und JavaDoc.
Kann-Ziele	Weitere Funktionen wurden im aktuellen Repository nicht umgesetzt. Mögliche Erweiterungen werden im Fazit genannt.
Nicht-Ziele	Kein Frontend, keine automatische Befüllung der Datenbank und keine Authentifizierung im vorhandenen Projektstand.

2. Anforderungskatalog

2.1 Übersicht der User Stories

ID	User Story	Priorität	Zugehörige Endpoints
US-01	Als API-Benutzer möchte ich einen neuen South-Park-Character erstellen können, damit dieser dauerhaft gespeichert und später abgerufen werden kann.	Muss	POST /api/characters; GET /api/characters/{id}
US-02	Als API-Benutzer möchte ich gespeicherte South-Park-Characters anzeigen können, damit ich die vorhandenen Daten und Details eines Characters sehe.	Muss	GET /api/characters; GET /api/characters/{id}
US-03	Als API-Benutzer möchte ich einem bestehenden Character ein Quote hinzufügen können, damit mehrere Zitate pro Character gespeichert werden können.	Muss	POST und GET /api/characters/{characterId}/quotes

2.2 User Story 1: Character erstellen

Als API-Benutzer möchte ich einen neuen South-Park-Character erstellen können, damit dieser in PostgreSQL gespeichert und später wieder abgerufen werden kann.

Akzeptanzkriterien

Nr.	Akzeptanzkriterium	Prüfung / Nachweis
AK-1	Ein Character kann über POST /api/characters erstellt werden.	MT1; CharacterController
AK-2	Name, Gender und Status dürfen nicht leer sein; Age darf nicht negativ sein.	MT6; CharacterControllerTest
AK-3	Gültige Daten führen zu HTTP 201 Created und einem CharacterDTO mit ID.	MT1
AK-4	Ungültige Daten führen zu HTTP 400 Bad Request mit ErrorResponseDTO.	MT6; automatisierter Controller-Test
AK-5	Der Character wird dauerhaft in PostgreSQL gespeichert.	CharacterRepositoryTest; GET nach POST

2.3 User Story 2: Characters anzeigen

Als API-Benutzer möchte ich gespeicherte South-Park-Characters anzeigen können, damit ich alle vorhandenen Characters und die Details eines einzelnen Characters abrufen kann.

Akzeptanzkriterien

Nr.	Akzeptanzkriterium	Prüfung / Nachweis
AK-1	Alle Characters können über GET /api/characters abgerufen werden.	MT2
AK-2	Ein Character kann über GET /api/characters/{id} abgerufen werden.	CharacterServiceTest; User-Story-Ablauf
AK-3	Eine leere Datenbank führt bei der Listenabfrage zu einer leeren JSON-Liste.	Installationshinweis / API-Verhalten
AK-4	Eine unbekannte ID führt zu HTTP 404 Not Found mit ErrorResponseDTO.	MT7; CharacterServiceTest

Nr.	Akzeptanzkriterium	Prüfung / Nachweis
AK-5	Die API gibt CharacterDTOs und keine Entity-Klassen zurück.	CharacterMapper / Controller-Antworten

2.4 User Story 3: Quote zu Character hinzufügen

Als API-Benutzer möchte ich einem bestehenden South-Park-Character ein Quote hinzufügen können, damit mehrere Zitate zu diesem Character gespeichert werden können.

Akzeptanzkriterien

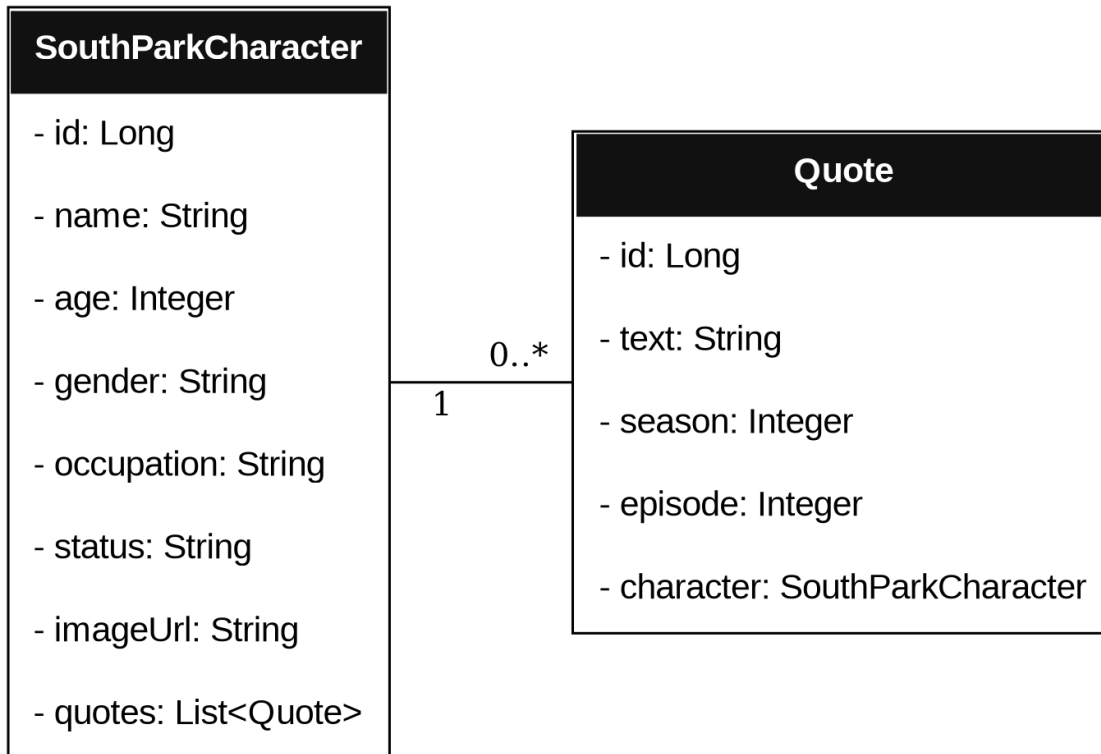
Nr.	Akzeptanzkriterium	Prüfung / Nachweis
AK-1	Ein Quote kann über POST /api/characters/{characterId}/quotes erstellt werden.	MT4
AK-2	Das Quote wird einem bestehenden Character zugeordnet; ein Character kann mehrere Quotes besitzen.	MT4 und Datenmodell
AK-3	Quotes eines Characters können über GET /api/characters/{characterId}/quotes geladen werden.	MT5
AK-4	Eine unbekannte Character-ID führt zu HTTP 404 Not Found.	ResourceNotFoundException / User-Story-Fehlerfall
AK-5	Leerer Text oder Season/Episode kleiner als 1 führen zu HTTP 400 Bad Request.	QuoteFormDTO / User-Story-Fehlerfall
AK-6	Die Antwort ist ein QuoteDTO mit characterId und keine Entity-Klasse.	QuoteMapper / MT4

2.5 Funktionale und nicht-funktionale Anforderungen

ID	Typ	Anforderung	Abnahmekriterium
F-01	Funktional	Vollständiges CRUD für SouthParkCharacter.	GET, POST, PUT und DELETE /api/characters funktionieren.
F-02	Funktional	CRUD für Quote und Verwaltung der Character-Beziehung.	Quote-Endpunkte und characterId funktionieren.
F-03	Funktional	Bean Validation für Character- und Quote-Eingaben.	Ungültige Daten ergeben HTTP 400.
F-04	Funktional	Dauerhafte Speicherung in PostgreSQL.	Daten werden über JPA gespeichert und erneut geladen.
NF-01	Nicht-funktional	Saubere 3-Schichten-Architektur.	Controller -> Service -> Repository.
NF-02	Nicht-funktional	DTOs und Mapper statt Entity-Ausgabe.	CharacterDTO und QuoteDTO werden nach aussen verwendet.
NF-03	Nicht-funktional	Robuste, einheitliche Fehlerbehandlung.	ErrorResponseDTO für 400 und 404.
NF-04	Nicht-funktional	Testbarkeit und dokumentierter Code.	6 automatisierte Tests, manuelle Tests und JavaDoc.

3. Klassendiagramm / Datenmodell / Architektur

3.1 Klassendiagramm



3.2 Haupt-Entity

Attribut	Datentyp	JPA / Validierung	Beschreibung
id	Long	@Id, @GeneratedValue(IDENTITY)	Eindeutige Datenbank-ID.
name	String	@Column(nullable=false, length=100); @NotBlank; @Size(max=100)	Name des Characters.
age	Integer	@Min(0)	Alter; darf nicht negativ sein.
gender	String	@Column(nullable=false, length=50); @NotBlank; @Size(max=50)	Geschlecht des Characters.
occupation	String	@Column(length=100); @Size(max=100)	Tätigkeit oder Rolle.

Attribut	Datentyp	JPA / Validierung	Beschreibung
status	String	@Column(nullable=false, length=50); @NotBlank; @Size(max=50)	Status des Characters.
imageUrl	String	@Column(name="image_url")	URL zu einem Bild.
quotes	List<Quote>	@OneToMany(mappedBy="character")	0 bis viele zugeordnete Quotes.

3.3 Verknüpfte Entity

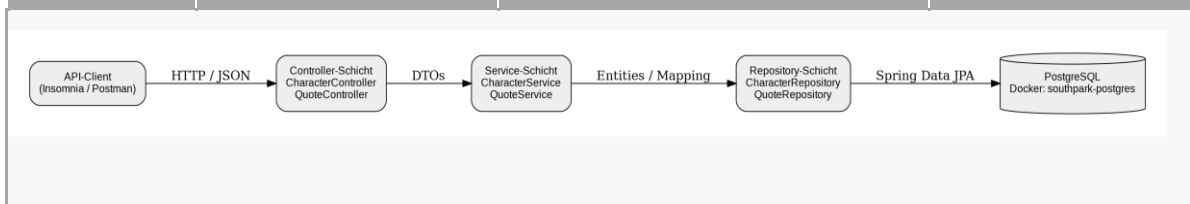
Attribut	Datentyp	JPA / Validierung	Beschreibung
id	Long	@Id, @GeneratedValue(IDENTITY)	Eindeutige Datenbank-ID.
text	String	@NotBlank; @Size(max=500)	Text des Zitats.
season	Integer	@Min(1)	Staffel; mindestens 1.
episode	Integer	@Min(1)	Episode; mindestens 1.
character	SouthParkCharacter	@ManyToOne; @JoinColumn(name="character_id", nullable=false)	Character, zu dem das Quote gehört.

3.4 Beziehungen im Datenmodell

Verbindung	Kardinalität	Umsetzung	Bedeutung
SouthParkCharacter -> Quote	1 -> 0..*	@OneToMany(mappedBy="character")	Ein Character kann kein, ein oder mehrere Quotes besitzen.
Quote -> SouthParkCharacter	* -> 1	@ManyToOne + character_id	Jedes Quote gehört genau zu einem bestehenden Character.

3.5 3-Schichten-Architektur

Schicht	Klassen / Pakete	Aufgabe	Nicht erlaubt
Controller	ch.wiss.southparkapi.controller: CharacterController, QuoteController	HTTP-Anfragen entgegennehmen, @Valid anwenden, DTOs und Statuscodes zurückgeben.	Geschäftslogik oder direkte Datenbankzugriffe.
Service	ch.wiss.southparkapi.service: CharacterService, QuoteService	Laden, Erstellen, Aktualisieren, Löschen und Mapping koordinieren.	Direkte HTTP-Verantwortung.
Repository	ch.wiss.southparkapi.repository: CharacterRepository, QuoteRepository	Datenzugriff über Spring Data JPA und JpaRepository.	Geschäftslogik.



3.6 DTO- und Mapper-Konzept

Element	Klasse	Verwendung
Response-DTO	CharacterDTO	Ausgabe bei Character-GET, POST und PUT; Entity wird nicht direkt ausgeliefert.
Form-DTO	CharacterFormDTO	Eingabe bei Character-POST und PUT mit Bean Validation.
Mapper	CharacterMapper	Wandelt zwischen SouthParkCharacter, CharacterFormDTO und CharacterDTO.
Response-DTO	QuoteDTO	Ausgabe für Quotes; enthält nur characterId statt der ganzen Character-Entity.
Form-DTO	QuoteFormDTO	Eingabe bei Quote-POST und PUT mit Bean Validation.
Mapper	QuoteMapper	Wandelt zwischen Quote, QuoteFormDTO und QuoteDTO.

3.7 Persistenz mit PostgreSQL und Docker

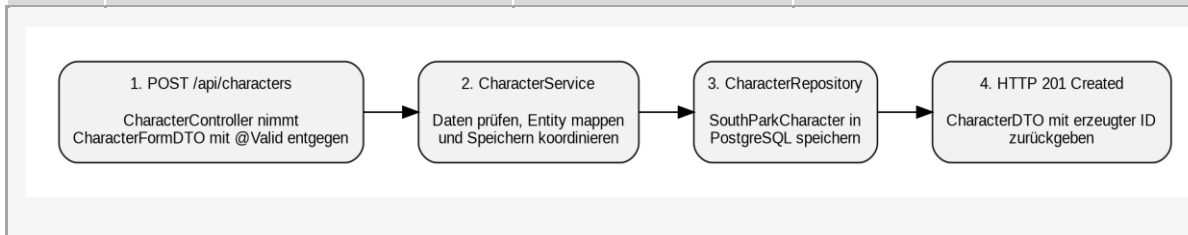
Eigenschaft	Wert / Beschreibung
Datenbank	PostgreSQL 16
Docker-Service	postgres; Container-Name southpark-postgres
Verbindung	localhost:5432, Datenbank southpark, Benutzer southpark
JPA	Entities und Spring Data Repositories; spring.jpa.hibernate.ddl-auto=update
Persistenznachweis	Manuelle API-Tests zeigen gespeicherte Characters und Quotes; Repository-Test speichert und lädt einen Character.

4. API-Abläufe je User Story

4.1 Ablauf User Story 1: Character erstellen

Kurzbeschreibung: Ein API-Client sendet gültige Character-Daten. Die Anwendung validiert, speichert und liefert den neu erstellten Character als DTO zurück.

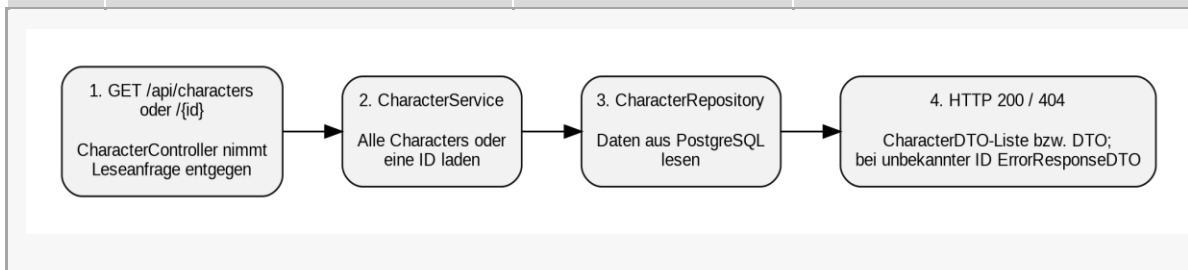
Schritt	Aktion / API-Call	Verarbeitende Schicht	Erwartetes Ergebnis
1	POST /api/characters mit CharacterFormDTO	CharacterController	Request wird mit @Valid entgegengenommen.
2	createCharacter(formDTO)	CharacterService	FormDTO wird in eine Entity umgewandelt.
3	characterRepository.save(character)	CharacterRepository	Character wird in PostgreSQL gespeichert.
4	CharacterDTO + HTTP 201 Created	CharacterController	Gespeicherte Daten inklusive ID werden zurückgegeben.



4.2 Ablauf User Story 2: Characters anzeigen

Kurzbeschreibung: Ein API-Client ruft alle Characters oder einen Character per ID ab. Die Daten werden aus PostgreSQL gelesen und als DTOs zurückgegeben.

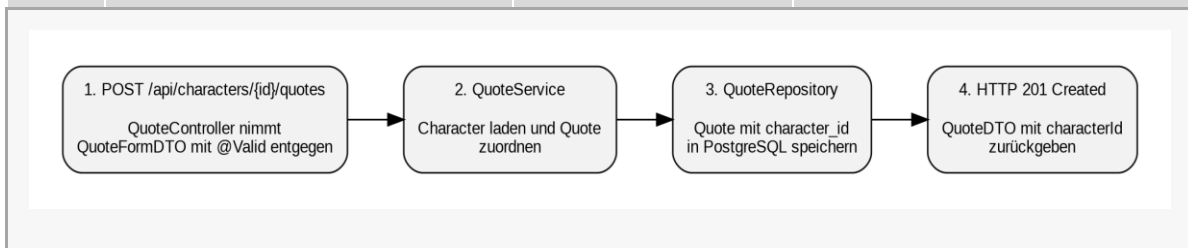
Schritt	Aktion / API-Call	Verarbeitende Schicht	Erwartetes Ergebnis
1	GET /api/characters oder /api/characters/{id}	CharacterController	Leseanfrage wird entgegengenommen.
2	getAllCharacters() oder getCharacterById(id)	CharacterService	Ladevorgang und Mapping werden koordiniert.
3	findAll() oder findById(id)	CharacterRepository	Daten werden aus PostgreSQL gelesen.
4	List<CharacterDTO> / CharacterDTO	CharacterController	HTTP 200; bei unbekannter ID HTTP 404 mit ErrorResponseDTO.



4.3 Ablauf User Story 3: Quote zu Character hinzufügen

Kurzbeschreibung: Ein API-Client sendet Quote-Daten für eine Character-ID. Der Character wird geprüft, das Quote zugeordnet und mit dem Fremdschlüssel `character_id` gespeichert.

Schritt	Aktion / API-Call	Verarbeitende Schicht	Erwartetes Ergebnis
1	POST /api/characters/{characterId}/quotes	QuoteController	QuoteFormDTO wird mit @Valid entgegengenommen.
2	createQuote(characterId, formDTO)	QuoteService	Character wird geladen und dem neuen Quote zugeordnet.
3	quoteRepository.save(quote)	QuoteRepository	Quote wird mit character_id in PostgreSQL gespeichert.
4	QuotedDTO + HTTP 201 Created	QuoteController	Antwort enthält ID, Text, Season, Episode und characterId.



5. DTOs / Datentypen / JSON-Beispiele

5.1 DTO-Übersicht

DTO	Richtung	Felder	Zweck
CharacterDTO	Response	id, name, age, gender, occupation, status, imageUrl	Ausgabe bei Character-GET, POST und PUT.
CharacterFormDTO	Request	name, age, gender, occupation, status, imageUrl	Eingabe bei Character-POST und PUT mit Bean Validation.
QuoteDTO	Response	id, text, season, episode, characterId	Kontrollierte Darstellung eines Quotes und seiner Beziehung.
QuoteFormDTO	Request	text, season, episode	Eingabe bei Quote-POST und PUT mit Bean Validation.
ErrorResponseDTO	Response	status, message, errors	Einheitliche Fehlerantwort für Validation und Not Found.

5.2 GET - Liste

Eigenschaft	Wert
Methode	GET
Endpoint	/api/characters
Erfolg	200 OK; List<CharacterDTO>
Fehler	Kein Fehler bei leerer Datenbank; es wird [] zurückgegeben.

Beispiel-Request

Kein Request-Body

Beispiel-Response

```
[
  {
    "id": 1,
    "name": "Eric Cartman",
    "age": 10,
    "gender": "Male",
    "occupation": "Student",
    "status": "Alive",
    "imageUrl": "https://example.com/cartman.png"
  }
]
```

5.3 GET - Einzelobjekt

Eigenschaft	Wert
Methode	GET
Endpoint	/api/characters/{id}

Eigenschaft	Wert
Erfolg	200 OK; CharacterDTO
Fehler	404 Not Found; ErrorResponseDTO

Beispiel-Request

Kein Request-Body

Beispiel-Response

```
{
  "id": 1,
  "name": "Eric Cartman",
  "age": 10,
  "gender": "Male",
  "occupation": "Student",
  "status": "Alive",
  "imageUrl": "https://example.com/cartman.png"
}
```

5.4 POST - Erfassen

Eigenschaft	Wert
Methode	POST
Endpoint	/api/characters
Erfolg	201 Created; CharacterDTO
Fehler	400 Bad Request; ErrorResponseDTO

Beispiel-Request

```
{
  "name": "Eric Cartman",
  "age": 10,
  "gender": "Male",
  "occupation": "Student",
  "status": "Alive",
  "imageUrl": "https://example.com/cartman.png"
}
```

Beispiel-Response

```
{
  "id": 1,
  "name": "Eric Cartman",
  "age": 10,
  "gender": "Male",
  "occupation": "Student",
  "status": "Alive",
  "imageUrl": "https://example.com/cartman.png"
}
```

5.5 PUT - Editieren

Eigenschaft	Wert
Methode	PUT
Endpoint	/api/characters/{id}
Erfolg	200 OK; aktualisiertes CharacterDTO
Fehler	400 Bad Request oder 404 Not Found; ErrorResponseDTO

Beispiel-Request

```
{
  "name": "Eric Cartman",
  "age": 10,
  "gender": "Male",
  "occupation": "Student",
  "status": "Alive",
  "imageUrl": "https://example.com/cartman-updated.png"
}
```

Beispiel-Response

```
{
  "id": 1,
  "name": "Eric Cartman",
  "age": 10,
  "gender": "Male",
  "occupation": "Student",
  "status": "Alive",
  "imageUrl": "https://example.com/cartman-updated.png"
}
```

5.6 DELETE - Löschen

Eigenschaft	Wert
Methode	DELETE
Endpoint	/api/characters/{id}
Erfolg	204 No Content
Fehler	404 Not Found; ErrorResponseDTO

Beispiel-Request

Kein Request-Body

Beispiel-Response

204 No Content - kein Response-Body

6. REST-Schnittstellen, Validierung und Fehlerbehandlung

6.1 Endpoint-Übersicht

Methode	Endpoint	Beschreibung	Request-DTO	Response / Status
GET	/api/characters	Alle Characters laden	-	200 / List<CharacterDTO>
GET	/api/characters/{id}	Character per ID laden	-	200 / CharacterDTO; 404
POST	/api/characters	Character erstellen	CharacterFormDTO + @Valid	201 / CharacterDTO; 400
PUT	/api/characters/{id}	Character aktualisieren	CharacterFormDTO + @Valid	200 / CharacterDTO; 400/404
DELETE	/api/characters/{id}	Character löschen	-	204; 404
GET	/api/quotes	Alle Quotes laden	-	200 / List<QuoteDTO>
GET	/api/quotes/{id}	Quote per ID laden	-	200 / QuoteDTO; 404
GET	/api/characters/{characterId}/quotes	Quotes eines Characters laden	-	200 / List<QuoteDTO>; 404
POST	/api/characters/{characterId}/quotes	Quote für Character erstellen	QuoteFormDTO + @Valid	201 / QuoteDTO; 400/404
PUT	/api/quotes/{id}	Quote aktualisieren	QuoteFormDTO + @Valid	200 / QuoteDTO; 400/404
DELETE	/api/quotes/{id}	Quote löschen	-	204; 404

6.2 Bean Validation

Feld	Annotation	Regel	Fehlermeldung
name	@NotBlank, @Size(max=100)	Nicht leer; höchstens 100 Zeichen.	Name must not be empty / ...no more than 100 characters
age	@Min(0)	Mindestens 0.	Age must be at least 0
gender	@NotBlank, @Size(max=50)	Nicht leer; höchstens 50 Zeichen.	Gender must not be empty / ...no more than 50 characters
occupation	@Size(max=100)	Höchstens 100 Zeichen.	Occupation must contain no more than 100 characters
status	@NotBlank, @Size(max=50)	Nicht leer; höchstens 50 Zeichen.	Status must not be empty / ...no more than 50 characters
text	@NotBlank, @Size(max=500)	Nicht leer; höchstens 500 Zeichen.	Quote text must not be empty / ...no more than 500 characters
season	@Min(1)	Mindestens 1.	Season must be at least 1
episode	@Min(1)	Mindestens 1.	Episode must be at least 1

6.3 Exception- und Handler-Konzept

Komponente	Klasse	Aufgabe
Eigene Exception	ResourceNotFoundException	Wird ausgelöst, wenn ein Character oder Quote mit der angeforderten ID nicht existiert.

Komponente	Klasse	Aufgabe
Globaler Handler	GlobalExceptionHandler	Fängt Validation- und Not-Found-Fehler zentral ab und setzt den HTTP-Status.
Fehler-DTO	ErrorResponseDTO	Liefert status, message und errors als einheitliche JSON-Antwort.

Beispiel einer wohlgeformten Fehlerantwort:

```
{
  "status": 404,
  "message": "Character with id 99999 was not found",
  "errors": []
}
```

6.4 HTTP-Statuscodes

Status	Bedeutung	Einsatz im Projekt
200 OK	Erfolgreich gelesen oder aktualisiert	GET und PUT
201 Created	Datensatz erfolgreich erstellt	POST
204 No Content	Datensatz erfolgreich gelöscht	DELETE
400 Bad Request	Bean Validation fehlgeschlagen	Ungültiges CharacterFormDTO oder QuoteFormDTO
404 Not Found	Character oder Quote nicht vorhanden	Unbekannte ID / fehlende Character-Zuordnung

7. Testplan

ID	Titel	Vorbedingung	Schritte / Request	Erwartetes Ergebnis
MT1	Character erstellen	App und PostgreSQL laufen; gültiger JSON-Body	POST /api/characters	201 Created; CharacterDTO mit ID
MT2	Alle Characters anzeigen	Mindestens ein Character vorhanden	GET /api/characters	200 OK; JSON-Liste
MT3	Character bearbeiten	Character-ID existiert	PUT /api/characters/{id}	200 OK; aktualisierte imageUrl
MT4	Quote zu Character erstellen	Character-ID existiert	POST /api/characters/{characterId}/quotes	201 Created; QuoteDTO mit characterId
MT5	Quotes eines Characters anzeigen	Character besitzt Quote	GET /api/characters/{characterId}/quotes	200 OK; Quote-Liste
MT6	Validation Error prüfen	Ungültiger Character-Body	POST /api/characters	400 Bad Request; strukturierte errors-Liste
MT7	Not Found Error prüfen	ID 99999 existiert nicht	GET /api/characters/99999	404 Not Found; ErrorResponseDTO
MT8	Character löschen	Character ohne Quotes existiert	DELETE /api/characters/{id}	204 No Content

7.1 Ergänzende projektspezifische Testfälle

ID	Testidee	Erwartetes Ergebnis
MT4/MT 5	Beziehung zwischen Character und Quote prüfen.	Quote wird mit characterId gespeichert und über den Character-Endpunkt geladen.
MT6/MT 7	Validation und zentrale Fehlerbehandlung prüfen.	HTTP 400 bzw. 404 mit strukturierter ErrorResponseDTO.

8. Testergebnisse

ID	Testfall	Soll-Ergebnis	Ist-Ergebnis	Status	Nachweis
MT1	Character erstellen	201 Created	Character wurde erstellt und erhielt eine ID.	Bestanden	Insomnia-Screenshot
MT2	Alle Characters anzeigen	200 OK; DTO-Liste	Liste mit gespeicherten Characters wurde geliefert.	Bestanden	Insomnia-Screenshot
MT3	Character bearbeiten	200 OK	Character und imageUrl wurden aktualisiert.	Bestanden	Insomnia-Screenshot
MT4	Quote erstellen	201 Created	Quote wurde Character 6 zugeordnet.	Bestanden	Insomnia-Screenshot
MT5	Quotes anzeigen	200 OK; Quote-Liste	Zugeordnetes Quote wurde geladen.	Bestanden	Insomnia-Screenshot
MT6	Validation	400 ErrorResponseDTO	Status 400, Validation failed und errors-Liste.	Bestanden	Insomnia-Screenshot
MT7	Unbekannte ID	404 ErrorResponseDTO	Strukturierte 404-Antwort wurde geliefert.	Bestanden	Insomnia-Screenshot
MT8	Character löschen	204 No Content	Character wurde ohne Response-Body gelöscht.	Bestanden	Insomnia-Screenshot

8.1 Zusammenfassung der manuellen Tests

Alle acht manuellen Tests wurden erfolgreich abgeschlossen. Die dokumentierten HTTP-Statuscodes und JSON-Antworten entsprachen den Erwartungen. Getestet wurden das Character-CRUD, das Erstellen und Laden von Quotes, die 1:n-Beziehung, Bean Validation sowie die zentrale 404-Fehlerbehandlung. In den bereitgestellten Testnachweisen sind keine verbleibenden Abweichungen dokumentiert.

Screenshot einfügen: MT1_POST_character_created

The screenshot shows the Insomnia client interface. At the top, there are tabs for different HTTP methods: POST, PUT, DELETE, and GET. The selected method is POST, and the endpoint is `http://localhost:8080/api/characters`. The request body is a JSON object:

```
1 {
2   "name": "Eric Cartman",
3   "age": 10,
4   "gender": "Male",
5   "occupation": "Student",
6   "status": "Alive",
7   "imageUrl": "https://example.com/cartman.png"
8 }
```

The response is a 201 Created status, with a response time of 231 ms and a body size of 140 B. The response body is a JSON object:

```
1 {
2   "id": 6,
3   "name": "Eric Cartman",
4   "age": 10,
5   "gender": "Male",
6   "occupation": "Student",
7   "status": "Alive",
8   "imageUrl": "https://example.com/cartman.png"
9 }
```

Screenshot einfügen: MT2_GET_all_characters

POST create character × PUT Put × DEL Delete × GET Get × +

My first project > My first collection > GET Get </> Base Environment Cookies Certificates

GET http://localhost:8080/api/characters Send 200 OK 111 ms 291 B Just Now

Params Body Auth Headers 4 Scripts Docs Preview Headers 3 Cookies Tests 0/0 → Mock Console

JSON

```
1 ...
```

Preview

```
1 [
2   {
3     "id": 2,
4     "name": "Eric Cartman",
5     "age": 10,
6     "gender": "Male",
7     "occupation": "Student",
8     "status": "Alive",
9     "imageUrl": "https://example.com/cartman.png"
10  },
11  {
12    "id": 6,
13    "name": "Eric Cartman",
14    "age": 10,
15    "gender": "Male",
16    "occupation": "Student",
17    "status": "Alive",
18    "imageUrl": "https://example.com/cartman-updated.png"
19  }
20 ]
```

Screenshot einfügen: MT3_PUT_character_updated

POST create character × PUT Put × DEL Delete × GET Get × +

My first project > My first collection > PUT Put </> Base Environment Cookies Certificates

PUT http://localhost:8080/api/characters/6 Send 200 OK 29 ms 148 B 1 Minute Ago

Params Body Auth Headers 4 Scripts Docs Preview Headers 3 Cookies Tests 0/0 → Mock Console

JSON

```
1 {
2   "name": "Eric Cartman",
3   "age": 10,
4   "gender": "Male",
5   "occupation": "Student",
6   "status": "Alive",
7   "imageUrl": "https://example.com/cartman-updated.png"
8 }
9
```

Preview

```
1 {
2   "id": 6,
3   "name": "Eric Cartman",
4   "age": 10,
5   "gender": "Male",
6   "occupation": "Student",
7   "status": "Alive",
8   "imageUrl": "https://example.com/cartman-updated.png"
9 }
```

Screenshot einfügen: MT4_POST_quote_for_character

The screenshot shows a Postman interface with a POST request to `http://localhost:8080/api/characters/6/quotes`. The request body is a JSON object: `{ "text": "Respect my authority!", "season": 2, "episode": 4 }`. The response is a 201 Created status with a body: `{ "id": 2, "text": "Respect my authority!", "season": 2, "episode": 4, "characterId": 6 }`.

Method	URL	Status	Time	Size
POST	http://localhost:8080/api/characters/6/quotes	201 Created	54 ms	78 B

```
1 {
2   "text": "Respect my authority!",
3   "season": 2,
4   "episode": 4
5 }
6
```

```
1 {
2   "id": 2,
3   "text": "Respect my authority!",
4   "season": 2,
5   "episode": 4,
6   "characterId": 6
7 }
```

Screenshot einfügen: MT5_GET_quotes_by_character

The screenshot shows a GET request to `http://localhost:8080/api/characters/6/quotes`. The response is a 200 OK status with a body: `[ { "id": 2, "text": "Respect my authority!", "season": 2, "episode": 4, "characterId": 6 } ]`.

Method	URL	Status	Time	Size
GET	http://localhost:8080/api/characters/6/quotes	200 OK	252 ms	80 B

```
1 ...
```

```
1 [
2   {
3     "id": 2,
4     "text": "Respect my authority!",
5     "season": 2,
6     "episode": 4,
7     "characterId": 6
8   }
9 ]
```

Screenshot einfügen: MT6_POST_character_validation_error

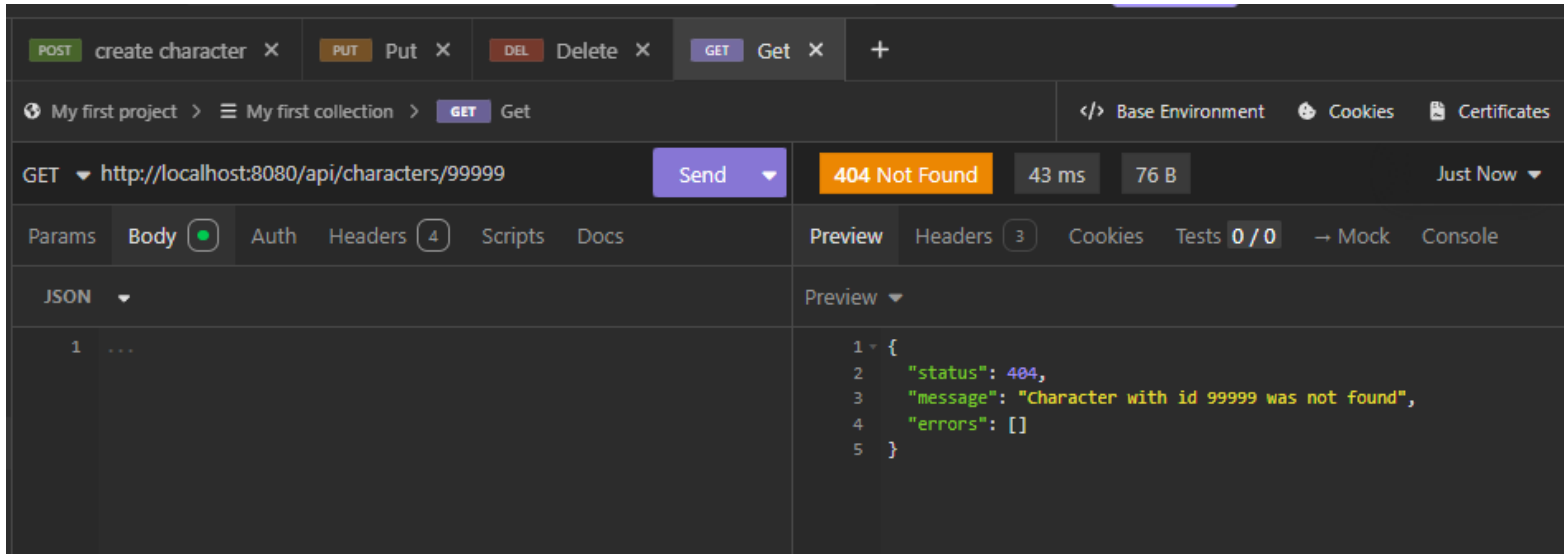
The screenshot shows a POST request to `http://localhost:8080/api/characters` with a body: `{ "name": "", "age": -1, "gender": "", "occupation": "Student", "status": "", "imageUrl": "https://example.com/cartman.png" }`. The response is a 400 Bad Request status with a body: `{ "status": 400, "message": "Validation failed", "errors": [ "Age must be at least 0", "Status must not be empty", "Gender must not be empty", "Name must not be empty" ] }`.

Method	URL	Status	Time	Size
POST	http://localhost:8080/api/characters	400 Bad Request	29 ms	159 B

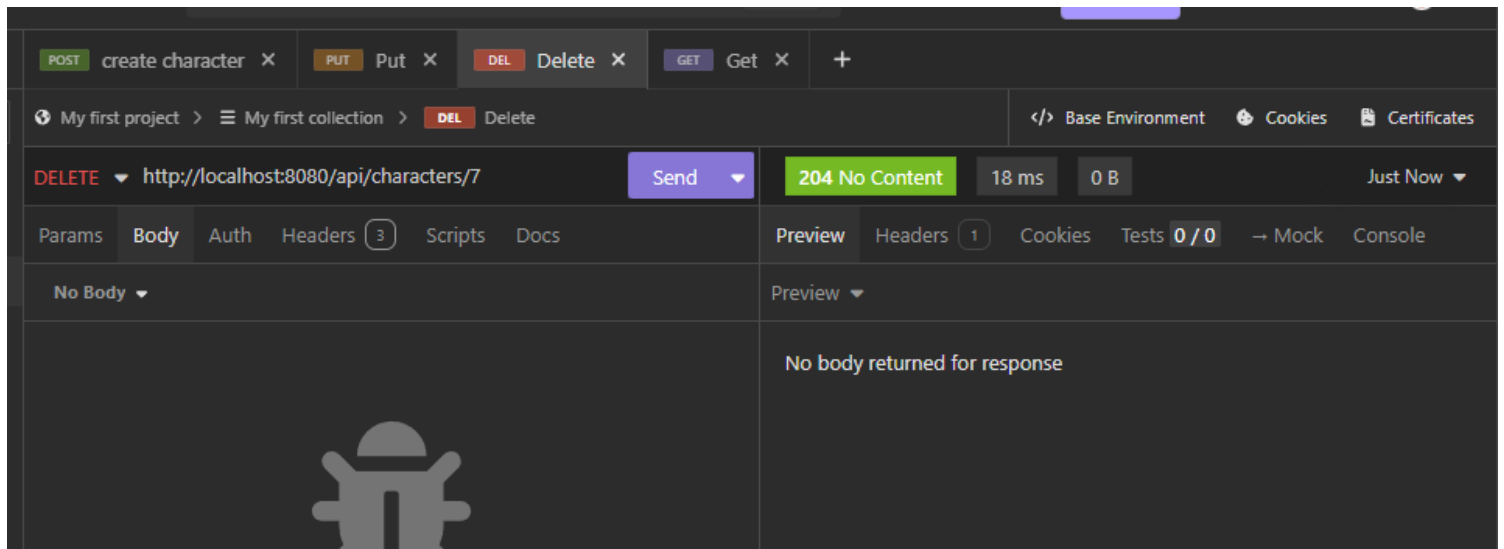
```
1 {
2   "name": "",
3   "age": -1,
4   "gender": "",
5   "occupation": "Student",
6   "status": "",
7   "imageUrl": "https://example.com/cartman.png"
8 }
9
10
```

```
1 {
2   "status": 400,
3   "message": "Validation failed",
4   "errors": [
5     "Age must be at least 0",
6     "Status must not be empty",
7     "Gender must not be empty",
8     "Name must not be empty"
9   ]
10 }
```

Screenshot einfügen: MT7_GET_character_not_found



Screenshot einfügen: MT8_DELETE_character



9. Automatisierte Tests und JavaDoc

9.1 Übersicht der automatisierten Tests

ID	Test-Typ	Testklasse / Methode	Prüfung	Erwartung
UT1	Repository	CharacterRepositoryTest.saveAndFindCharacterById	Character speichern und per ID laden.	Entity wird korrekt persistiert und gefunden.
UT2	Service	CharacterServiceTest.getCharacterByIdReturnsCharacter	Vorhandene Character-ID laden.	CharacterDTO wird korrekt zurückgegeben.
UT3	Service	CharacterServiceTest.getCharacterByIdThrowsExceptionWhenCharacterDoesNotExist	Unbekannte Character-ID.	ResourceNotFoundException wird ausgelöst.
UT4	Service	QuoteServiceTest.createQuoteAddsQuoteToCharacter	Quote für bestehenden Character erstellen.	QuoteDTO mit characterId wird zurückgegeben.
UT5	Controller	CharacterControllerTest.createCharacterReturnsBadRequestForInvalidData	Ungültiger POST-Request.	HTTP 400 Bad Request.
UT6	Context	SouthparkApiApplicationTests.contextLoads	Spring-Boot-Context laden.	Anwendungskontext startet erfolgreich.

9.2 Durchführung und Ergebnis

Befehl	Ergebnis	Datum
mvn test	6 Tests; 0 Failures; 0 Errors; 0 Skipped; BUILD SUCCESS	06.07.2026
mvn clean install	Build erfolgreich; Tests während des Builds erfolgreich	06.07.2026

```

2026-07-06T15:37:45.088+02:00 INFO 30468 --- [southpark-api] [main] c.w.s.SouthparkApiApplicationTests : Start
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.437 s -- in ch.wiss.southparkapi.SouthparkApiApplicationTests
[INFO] Results:
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
[INFO] Total time: 7.072 s
[INFO] Finished at: 2026-07-06T15:37:45+02:00
[INFO]
PS C:\Schule\Modul 295\southpark-api>

```

9.3 JavaDoc

Paket / Klasse	JavaDoc vorhanden	Bemerkung
Controller	Ja	CharacterController und QuoteController sind dokumentiert.
Service	Ja	CharacterService und QuoteService sind dokumentiert.
Repository	Ja	Eigene Repository-Interfaces sind dokumentiert; Getter/Setter bewusst ohne JavaDoc.
Entity / DTO / Mapper	Ja	SouthParkCharacter, Quote, DTOs und Mapper sind dokumentiert.
Exception / Handler	Ja	ResourceNotFoundException und GlobalExceptionHandler sind dokumentiert.

10. Installationsanleitung

10.1 Voraussetzungen

- Java JDK 21
- IntelliJ IDEA oder vergleichbare IDE
- Maven oder Maven Wrapper
- Docker Desktop / Docker Engine
- Git
- Insomnia oder Postman als REST-Client

10.2 Repository herunterladen

```
git clone https://github.com/silvio-apa/SouthPark-API.git
cd SouthPark-API
```

10.3 PostgreSQL-Datenbank starten

```
docker compose up -d
```

Einstellung	Wert
Host	localhost
Port	5432
Datenbank	southpark
Benutzer	southpark
Passwort	southpark

10.4 Projekt bauen und starten

```
mvn clean install
mvn spring-boot:run
```

10.5 Funktion prüfen

Nach dem Start der PostgreSQL-Datenbank und der Spring-Boot-Anwendung kann zuerst geprüft werden, ob die API erreichbar ist:

```
GET http://localhost:8080/api/characters
```

Wenn die Datenbank nach dem ersten Start noch leer ist, gibt die API eine leere JSON-Liste zurück:

```
[ ]
```

Danach kann ein Test-Character erstellt werden:

```
POST http://localhost:8080/api/characters
```

Request-Body:

```
{
  "name": "Eric Cartman",
  "age": 10,
  "gender": "Male",
  "occupation": "Student",
  "status": "Alive",
  "imageUrl": "https://example.com/cartman.png"
}
```

Erwartetes Ergebnis:

```
HTTP 201 Created
```

Die Response enthält den erstellten Character inklusive ID. Diese ID kann danach für weitere Tests verwendet werden.

Anschliessend kann ein Quote für den erstellten Character erstellt werden:

```
POST http://localhost:8080/api/characters/{characterId}/quotes
```

Dabei muss {characterId} durch die ID des vorher erstellten Characters ersetzt werden.

Request-Body:

```
{
  "text": "Respect my authority!",
  "season": 2,
  "episode": 4
}
```

Erwartetes Ergebnis:

```
HTTP 201 Created
```

Die Response enthält das erstellte Quote inklusive characterId.

Danach können die erstellten Daten mit folgenden Endpunkten geprüft werden:

```
GET http://localhost:8080/api/characters
GET http://localhost:8080/api/characters/{characterId}/quotes
```

10.6 Tests ausführen

```
mvn test
```

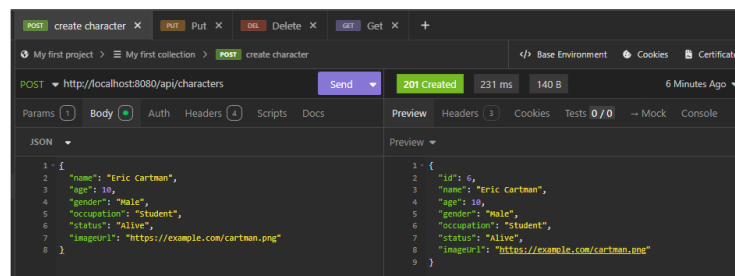
10.7 Troubleshooting

Problem	Mögliche Ursache	Lösung
Datenbank nicht erreichbar	Docker-Container gestoppt oder Port 5432 belegt	Docker Desktop starten; docker compose ps prüfen; Portbelegung kontrollieren.
Port 8080 belegt	Andere Anwendung verwendet den Port	Andere Anwendung beenden oder Server-Port anpassen.
Maven-Build schlägt fehl	Falsche JDK-Version oder nicht geladene Dependencies	JDK 21 prüfen und Maven-Projekt neu laden.
GET liefert []	Datenbank ist nach dem ersten Start leer	Zuerst einen Character mit POST /api/characters erstellen.

10.8 Start- und Installationsnachweis

```
2026-07-06T15:37:45.088+02:00 INFO 39488 --- [southpark-api] [main] ch.wiss.southparkapi.ApplicationTests : Started SouthparkApiApplicationTests
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.437 s -- in ch.wiss.southparkapi.SouthparkApiApplicationTests
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 7.072 s
[INFO] Finished at: 2026-07-06T15:37:45+02:00
[INFO]
PS C:\Schule\Modul 295\southpark-api>
```

Maven-Testlauf: 6 Tests, BUILD SUCCESS



API-Funktionsnachweis: POST /api/characters - 201 Created

11. Hilfestellungen / Quellen

Quelle / Person	Art der Hilfe
Projektpartner Silvio Apa / Nicola Schmied	Gemeinsame Projektarbeit gemäss Teamangabe der Vorlage.
Kursunterlagen / Dokumentationsvorlage	Struktur und Bewertungskriterien für Modul 295.
GitHub-Repository SouthPark-API	Primärquelle für Projektstruktur, Klassen, Validierung, Konfiguration und Commits.
Bereitgestellte Projektdokumente und Screenshots	Installations-, Architektur-, User-Story- sowie Testnachweise.
KI-Tool: ChatGPT	- Unterstützung bei Struktur, Formulierungen, Abgleich und Dokumentationsentwurf. - Unterstützung bei Fehler Behebung im Code. - Generell wurde die KI für Fehler Behebung und bei Stecken bleiben im Prozess zugezogen.

12. Projektverlauf / Git-Pushes

Push	Datum / Uhrzeit	Commit-Titel	Umgesetzte Arbeiten
1	04.07.2026 09:52 Uhr	Initialize Spring Boot project	• Spring-Boot-Projekt mit Maven, Java 21 und Spring Boot 4.x erstellt. • Dependencies für Spring Web, Spring Data JPA, PostgreSQL Driver und Validation hinzugefügt. • PostgreSQL über Docker Compose konfiguriert. • Datenbankverbindung in application.properties eingerichtet. • Anwendung erfolgreich mit PostgreSQL verbunden.
2	04.07.2026 10:38 Uhr	Add character entity and repository	• Entity SouthParkCharacter mit den Feldern id, name, age, gender, occupation, status und imageUrl erstellt. • JPA-Annotationen für Entity, Tabelle, Primärschlüssel und Spalten ergänzt. • CharacterRepository als JpaRepository für SouthParkCharacter erstellt. • Datenbanktabelle south_park_character automatisch in PostgreSQL erstellt.
3	04.07.2026 11:55 Uhr	Add character DTO mapper service and endpoints	• CharacterDTO und CharacterFormDTO für die API-Datenstruktur erstellt. • CharacterMapper zum Umwandeln zwischen Entity und DTO ergänzt. • CharacterService mit Methoden zum Laden und Erstellen von Characters erstellt. • CharacterController mit REST-Endpunkten für GET und POST erstellt. • GET /api/characters gibt alle Characters als DTO-Liste zurück. • GET /api/characters/{id} gibt einen einzelnen Character als DTO zurück. • POST /api/characters erstellt einen neuen Character in PostgreSQL.
4	04.07.2026 12:26 Uhr	Add character update delete endpoints and cleanup	• CharacterService um Methoden zum Aktualisieren und Löschen von Characters erweitert. • PUT /api/characters/{id} zum Bearbeiten eines Characters ergänzt. • DELETE /api/characters/{id} zum Löschen eines Characters ergänzt. • Character-CRUD für GET, POST, PUT und DELETE vollständig umgesetzt. • Code formatiert.
5	04.07.2026 12:50 Uhr	Add quote entity and character relationship	• Entity Quote mit den Feldern id, text, season und episode erstellt. • Many-to-One-Beziehung von Quote zu SouthParkCharacter ergänzt. • One-to-Many-Beziehung von SouthParkCharacter zu Quote ergänzt. • QuoteRepository als JpaRepository erstellt. • Methode findByCharacterId ergänzt, um Quotes eines bestimmten Characters zu laden. • Datenbanktabelle quote automatisch in PostgreSQL erstellt. • Fremdschlüssel character_id verbindet Quotes mit SouthParkCharacter.
6	04.07.2026 14:47 Uhr	Add quote DTO mapper service and endpoints	• QuoteDTO und QuoteFormDTO für die API-Datenstruktur erstellt. • QuoteMapper zum Umwandeln zwischen Quote-Entity und DTO ergänzt. • QuoteService mit Methoden zum Laden, Erstellen, Bearbeiten und Löschen von Quotes erstellt. • QuoteController mit REST-Endpunkten für das Quote-CRUD ergänzt. • GET /api/quotes gibt alle Quotes als DTO-Liste zurück. • GET /api/quotes/{id} gibt ein einzelnes Quote als DTO zurück. • GET /api/characters/{characterId}/quotes gibt alle Quotes eines Characters zurück. • POST /api/characters/{characterId}/quotes erstellt ein neues Quote für einen bestehenden Character. • PUT /api/quotes/{id} aktualisiert ein bestehendes Quote. • DELETE /api/quotes/{id} löscht ein bestehendes Quote. • Quote-Endpunkte erfolgreich mit Insomnia getestet. • Code formatiert und unnötige Imports entfernt.
7	04.07.2026 15:47 Uhr	Add validation exception handling and import cleanup	• Bean Validation in CharacterFormDTO und QuoteFormDTO ergänzt. • @Valid in CharacterController und QuoteController eingebaut. • ResourceNotFoundException für nicht gefundene Ressourcen erstellt. • ErrorResponseDTO für strukturierte Fehlerantworten erstellt. • GlobalExceptionHandler für zentrale Fehlerbehandlung ergänzt. • Validierungsfehler werden als 400 Bad Request mit strukturierter JSON-Antwort zurückgegeben. • Nicht gefundene Characters und Quotes werden als 404 Not Found mit strukturierter JSON-Antwort zurückgegeben. • Unnötige Imports aufgeräumt.

8	06.07.2026 14:38 Uhr	Add repository service and controller tests	• CharacterRepositoryTest zum Speichern und Laden eines SouthParkCharacter über JPA erstellt. • CharacterServiceTest zum Laden eines Characters anhand seiner ID erstellt. • CharacterServiceTest um einen Fehlerfall mit ResourceNotFoundException erweitert. • QuoteServiceTest zum Erstellen eines Quotes für einen bestehenden Character erstellt. • CharacterControllerTest zur Prüfung der Validierung bei ungültigen POST-Daten erstellt. • Repository-, Service- und Controller-Testtypen umgesetzt. • Alle Tests mit mvn test erfolgreich ausgeführt.
9	06.07.2026 14:45 Uhr	Add missing JavaDoc comments and verify build	• Fehlende JavaDoc-Kommentare bei der Main-Klasse und der Spring-Boot-Testklasse ergänzt. • JavaDoc-Kommentare der Projektklassen überprüft. • Getter und Setter bewusst ohne JavaDoc belassen. • Projekt mit Maven gebaut. • Alle Tests während des Builds erfolgreich ausgeführt. • Build ohne Kompilierfehler abgeschlossen.
10	11.07.2026	Final Push	• Dokumentation • README

13. Fazit

13.1 Zielerreichung

Die drei dokumentierten User Stories wurden vollständig umgesetzt: Characters können erstellt und angezeigt werden, und Quotes können einem bestehenden Character zugeordnet werden. Darüber hinaus sind PUT- und DELETE-Endpunkte für beide Ressourcen vorhanden. Die 1:n-Beziehung ist in JPA und PostgreSQL abgebildet. Bean Validation, strukturierte 400-/404-Antworten, DTOs, Mapper, Docker-Konfiguration, manuelle Tests und sechs automatisierte Tests sind nachgewiesen. Im gelieferten Projektstand sind keine offenen funktionalen Fehler dokumentiert.

13.2 Technische Erkenntnisse

Das Projekt zeigt den vollständigen Weg einer HTTP-Anfrage durch Controller, Service und Repository bis zur PostgreSQL-Datenbank. Die JPA-Beziehung verbindet Character und Quote über `character_id`. DTOs und Mapper schützen das externe API-Modell vor direkter Entity-Ausgabe und verhindern beim Quote die rekursive Ausgabe des gesamten Characters. Bean Validation prüft FormDTOs bereits am Controller. `ResourceNotFoundException` und `GlobalExceptionHandler` sorgen für einheitliche Fehlerantworten. Repository-, Service-, Controller- und Context-Tests zeigen, wie unterschiedliche Ebenen gezielt geprüft werden können.

13.3 Herausforderungen und Lösungen

Zu den zentralen technischen Aufgaben gehörten die Docker-/PostgreSQL-Verbindung, die bidirektionale Entity-Beziehung und die kontrollierte JSON-Ausgabe. Die Datenbankverbindung wurde über `docker-compose.yml` und `application.properties` vereinheitlicht. Die Beziehung wurde mit `@OneToMany` und `@ManyToOne` umgesetzt. Um rekursive oder zu umfangreiche Antworten zu vermeiden, enthält `QuoteDTO` lediglich die `characterId`. Validation- und Not-Found-Fälle wurden über einen zentralen Handler vereinheitlicht. Abschliessend wurden Tests ergänzt und der Maven-Build überprüft.

14. Arbeitsaufteilung

Die SouthPark API wurde grundsätzlich gemeinsam entwickelt. Die einzelnen Aufgaben wurden nicht streng zwischen den Teammitgliedern aufgeteilt. Stattdessen wurden die nächsten Schritte jeweils gemeinsam geplant, umgesetzt und überprüft.

Beide Teammitglieder arbeiteten an der Projektstruktur, den Entities SouthParkCharacter und Quote, der 1:n-Beziehung sowie an den Repositories, Services und Controllern mit. Auch die DTOs, Mapper, Validierungen und die zentrale Fehlerbehandlung wurden gemeinsam umgesetzt.

Die PostgreSQL-Datenbank und die Docker-Konfiguration wurden ebenfalls zusammen eingerichtet und getestet. Während der Entwicklung wurden auftretende Fehler gemeinsam analysiert und gelöst. Dadurch waren beide Teammitglieder jederzeit über den aktuellen Stand des Projekts informiert.

Auch die manuellen Tests mit Insomnia, die automatisierten Tests und die Projektdokumentation wurden gemeinsam erstellt und kontrolliert. Zusammenfassend waren beide Teammitglieder an der Planung, Programmierung, Fehlerbehebung, Durchführung der Tests und Dokumentation beteiligt